

Tugas 1 - Membuat Node Sederhana dan Menampilkan Datanya

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

node1 = Node(10)

print("Data Node:", node1.data)
```

Data Node: 10

Tugas 2 - Membuat Linked List Berisi 5 Angka

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)
node5 = Node(50)

node1.next = node2
node2.next = node3
node3.next = node4
node4.next = node5

head = node1
```

Tugas 3 - Traversal Seluruh Node

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)
node4 = Node(40)
node5 = Node(50)

node1.next = node2
node2.next = node3
node3.next = node4
node4.next = node5

current = node1

while current:
    print(current.data)
    current = current.next
```

10
20
30
40
50

Tugas 4 - Menambahkan Node di Awal Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(20)

node_baru = Node(10)
```

```
node_baru.next = head
head = node_baru

current = head

while current:
    print(current.data)
    current = current.next
```

10
20

Tugas 5 - Menambahkan Node di Akhir Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)

current = head

while current.next:
    current = current.next

current.next = Node(30)

current = head

while current:
    print(current.data)
    current = current.next
```

10
20
30

Tugas 6 - Menghapus Node Tertentu

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)

hapus = 20

current = head

while current.next:
    if current.next.data == hapus:
        current.next = current.next.next
        break
    current = current.next

current = head

while current:
    print(current.data)
    current = current.next
```

10
30

Tugas 7 - Mencari Data pada Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
head = Node(10)
head.next = Node(20)
head.next.next = Node(30)

cari = 20
ketemu = False

current = head

while current:
    if current.data == cari:
        ketemu = True
        break
    current = current.next

if ketemu:
    print("Data ditemukan")
else:
    print("Data tidak ditemukan")
```

Data ditemukan

Tugas 8 - Menghitung Jumlah Node

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = Node(10)
head.next = Node(20)
head.next.next = Node(30)
head.next.next.next = Node(40)

jumlah = 0
current = head

while current:
    jumlah += 1
    current = current.next

print("Jumlah Node:", jumlah)
```

Jumlah Node: 4

Tugas 9 - Implementasi Doubly Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

node1 = Node("A")
node2 = Node("B")
node3 = Node("C")

node1.next = node2
node2.prev = node1

node2.next = node3
node3.prev = node2

current = node1

while current:
    print(current.data)
    current = current.next
```

A
B
C

Tugas 10 - Implementasi Circular Linked List

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

node1 = Node(10)
node2 = Node(20)
node3 = Node(30)

node1.next = node2
node2.next = node3
node3.next = node1

current = node1

for i in range(6):
    print(current.data)
    current = current.next
```

10
20
30
10
20
30

Tugas 11 - Sistem Playlist Musik Menggunakan Linked List

```
class Lagu:
    def __init__(self, judul):
        self.judul = judul
        self.next = None

lagu1 = Lagu("Perfect")
lagu2 = Lagu("Photograph")
lagu3 = Lagu("Shape of You")

lagu1.next = lagu2
lagu2.next = lagu3

current = lagu1

print("Playlist Musik")

while current:
    print(current.judul)
    current = current.next
```

Playlist Musik
Perfect
Photograph
Shape of You

Tugas 12 - Browser History Menggunakan Doubly Linked List

```
class Halaman:
    def __init__(self, url):
        self.url = url
        self.prev = None
        self.next = None

google = Halaman("google.com")
youtube = Halaman("youtube.com")
github = Halaman("github.com")

google.next = youtube
youtube.prev = google

youtube.next = github
github.prev = youtube

current = github

print("Halaman saat ini:", current.url)

current = current.prev
print("Back ke:", current.url)
```

```
current = current.prev
print("Back ke:", current.url)
```

```
Halaman saat ini: github.com
Back ke: youtube.com
Back ke: google.com
```

Tugas 13 - Analisis Kompleksitas Operasi Linked List

Pada implementasi Doubly Linked List, operasi penelusuran data (traversal) dilakukan dengan mengunjungi setiap node satu per satu sehingga memiliki kompleksitas waktu $O(n)$. Penambahan node di awal linked list dapat dilakukan secara langsung dengan mengubah referensi head sehingga kompleksitasnya $O(1)$. Namun, jika penambahan dilakukan di akhir linked list tanpa menyimpan referensi tail, program harus menelusuri seluruh node terlebih dahulu sehingga kompleksitasnya menjadi $O(n)$.

Pada Circular Linked List, proses traversal juga memiliki kompleksitas $O(n)$ karena setiap node harus dikunjungi satu per satu. Penambahan node di awal atau akhir dapat dilakukan dalam $O(1)$ apabila tersedia referensi ke node terakhir (tail). Jika tidak ada referensi tail, pencarian node terakhir terlebih dahulu menyebabkan kompleksitas menjadi $O(n)$.

Pada sistem playlist musik yang menggunakan Linked List, proses menampilkan seluruh lagu dilakukan dengan traversal dari node pertama hingga node terakhir sehingga memiliki kompleksitas $O(n)$. Penambahan lagu di akhir playlist juga memerlukan pencarian node terakhir terlebih dahulu sehingga kompleksitasnya $O(n)$. Pencarian lagu tertentu juga membutuhkan traversal sehingga kompleksitasnya $O(n)$.

Pada browser history yang menggunakan Doubly Linked List, perpindahan ke halaman sebelumnya (Back) atau halaman berikutnya (Forward) dapat dilakukan secara langsung melalui pointer prev dan next. Oleh karena itu, operasi tersebut memiliki kompleksitas $O(1)$. Namun, jika pengguna ingin mencari halaman tertentu dalam riwayat browser, program tetap harus melakukan traversal sehingga kompleksitasnya menjadi $O(n)$.

Secara keseluruhan, operasi yang melibatkan akses langsung ke node tertentu seperti perpindahan menggunakan pointer next atau prev memiliki kompleksitas $O(1)$. Sebaliknya, operasi yang memerlukan pencarian atau penelusuran seluruh node memiliki kompleksitas $O(n)$ karena harus mengunjungi node satu per satu hingga data yang dicari ditemukan atau hingga akhir linked list tercapai.